EDA Design Flow Integration

Lecture 5: Parallelism & Pipelining

辛艾曼 Aman Sinha

Institute of Integrated Circuit Design

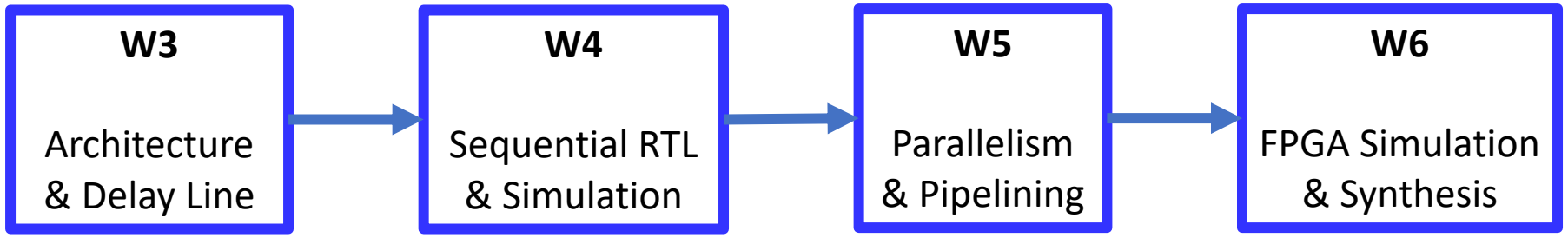
National Sun Yat-Sen University

Week 5 Learning Objectives

- Explain how parallelism and pipelining can improve hardware throughput
- Compare the Week 4 sequential FIR baseline with parallel and pipelined FIR organizations
- Describe the trade-off between time-multiplexed hardware reuse and hardware replication
- Identify pipeline registers and explain their effect on latency
- Track valid signals through multi-cycle and pipelined datapaths
- Read waveforms for parallel and pipelined FIR behavior
- Use simulation evidence before later FPGA and cell-based IC flows
- Prepare for Week 6 Xilinx FPGA simulation and synthesis

Where Week 5 Fits

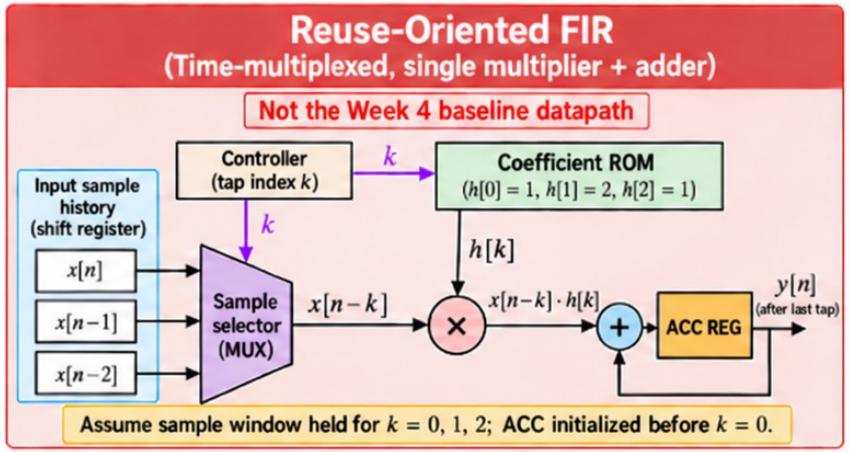
- Week 3 introduced FIR filter architecture and delay-line thinking
- Week 4 converted the FIR idea into sequential RTL and simulation evidence
- Week 5 studies architectural choices that change timing and resource use
- The same function can be implemented with different datapath structures
- Today prepares students to interpret later synthesis and implementation reports

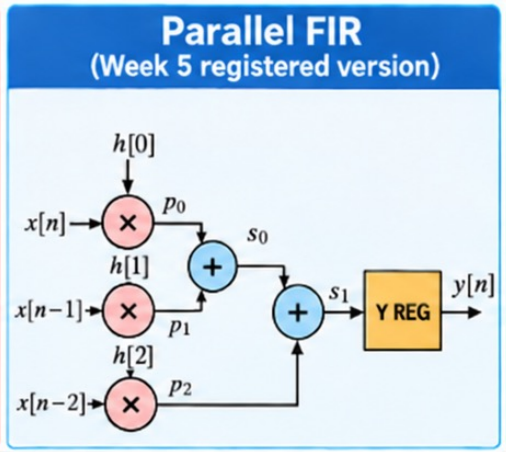


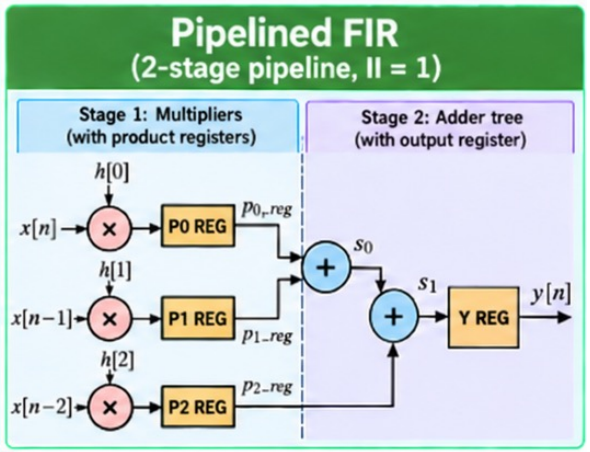
Architecture changes timing + resource expectations

Hardware Cost vs Time

- Architectural choices change both resource use and timing behavior
- Reuse-oriented time-multiplexing can reduce arithmetic hardware but may add cycles
- Parallel replication increases hardware but can improve throughput
- Pipelining inserts registers to shorten long combinational paths
- The best choice depends on design goals and constraints







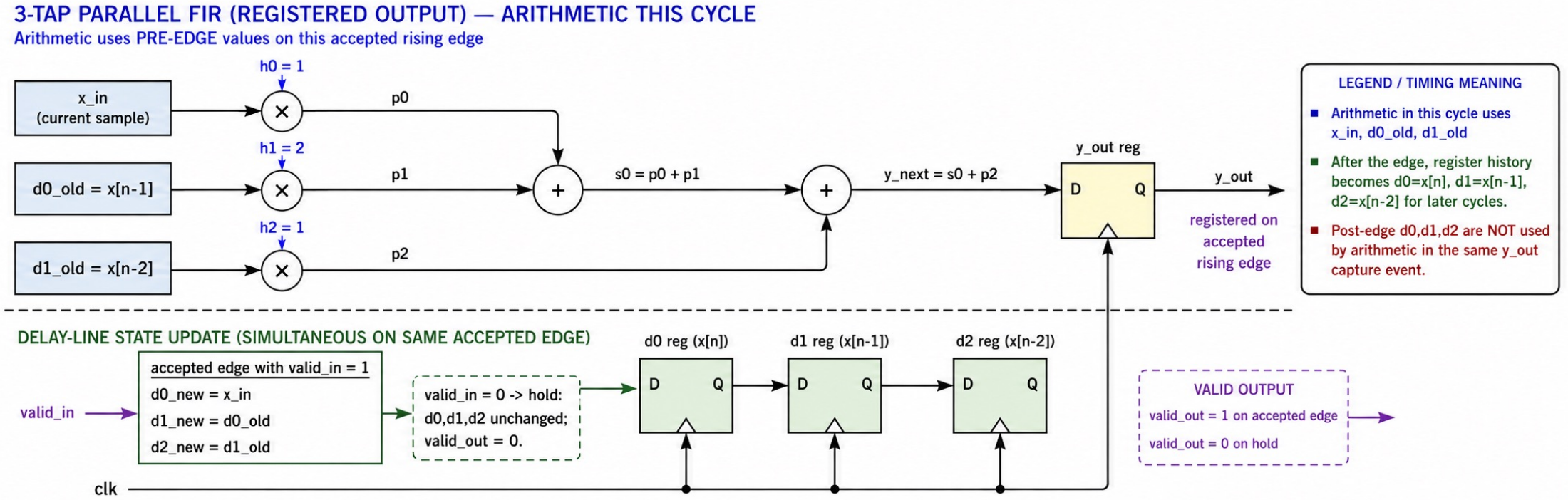
Multipliers	1	3	3
Adders	1	2 (combinational adder tree)	2 (adder tree in Stage 2)
Datapath registers shown (excluding input history/control)	1 (accumulator)	1 (output register)	4 (3 product registers + 1 output register)
Throughput	3 tap cycles/output (excluding control overhead)	Up to one output/cycle when one valid input is accepted per cycle	Up to one output/cycle after fill for an $II = 1$ pipeline
Combinational path each cycle	Less arithmetic replicated per cycle (time-multiplexed)	Multiplier(s) + adder tree (longer combinational path)	Shorter per-stage combinational path due to pipeline registers

- Reuse reduces arithmetic replication but increases cycles per output; parallelism increases concurrent hardware to improve throughput potential, while pipelining adds registers to improve timing potential.
- All architectures implement $y[n] = h[0]x[n] + h[1]x[n-1] + h[2]x[n-2]$ with $h[0] = 1, h[1] = 2, h[2] = 1$ (3-tap example).

3-Tap Parallel FIR Datapath

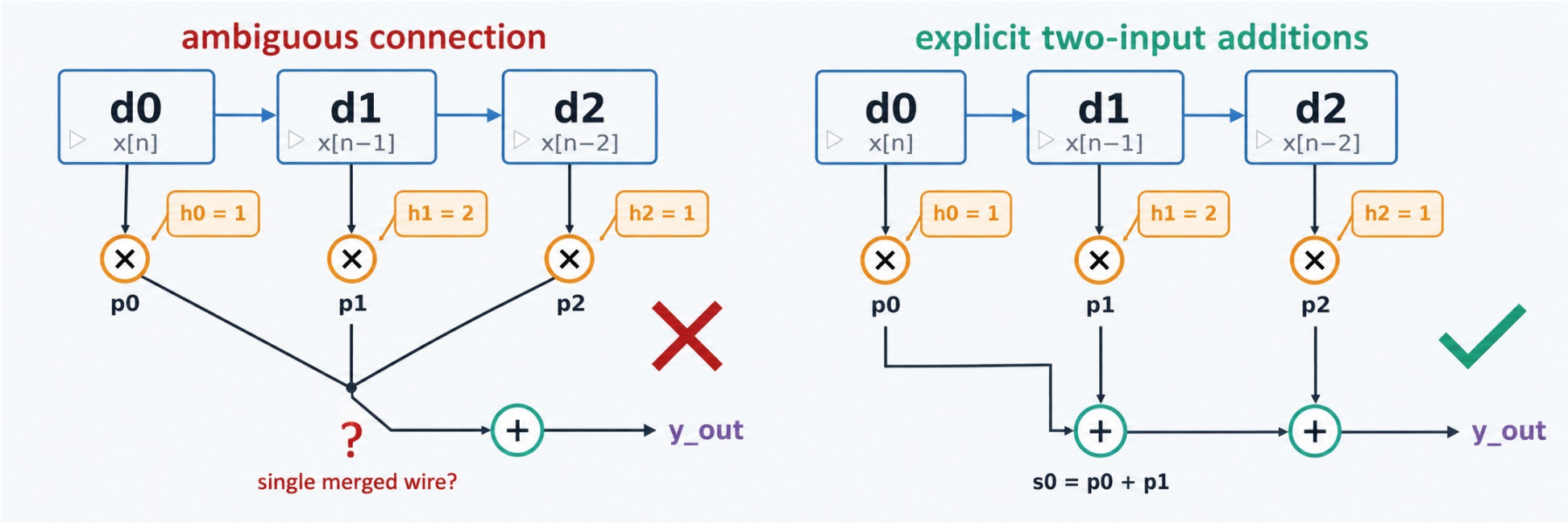
- The current input sample feeds tap 0
- Two delay registers hold the previous two samples
- Each tap is multiplied by its coefficient
- Three products enter an explicit adder structure
- The output is registered on the accepted rising edge

d2 is retained to show additional sample history; the 3-tap arithmetic itself uses x_in, d0, and d1.



Adder Structure

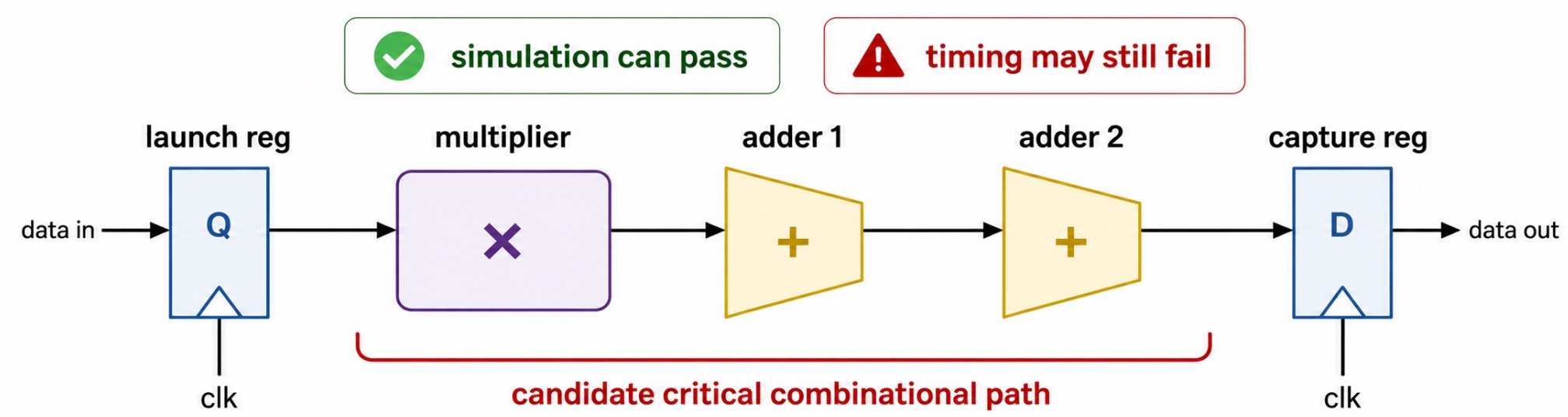
- Multiple products cannot be merged ambiguously onto one wire
- Adder inputs should be explicit in diagrams and RTL thinking
- A chain or tree of adders can implement the same mathematical sum
- The adder structure influences timing paths
- Later synthesis reports may reveal the cost of this structure



Critical Path Reminder

- The critical path is the timing path with the worst slack under the applied timing constraints
- A parallel FIR may place multipliers and adders in one cycle
- If the path is too long, timing may fail at the target clock
- Pipelining is a common way to shorten a long path
- Simulation correctness alone does not prove timing closure

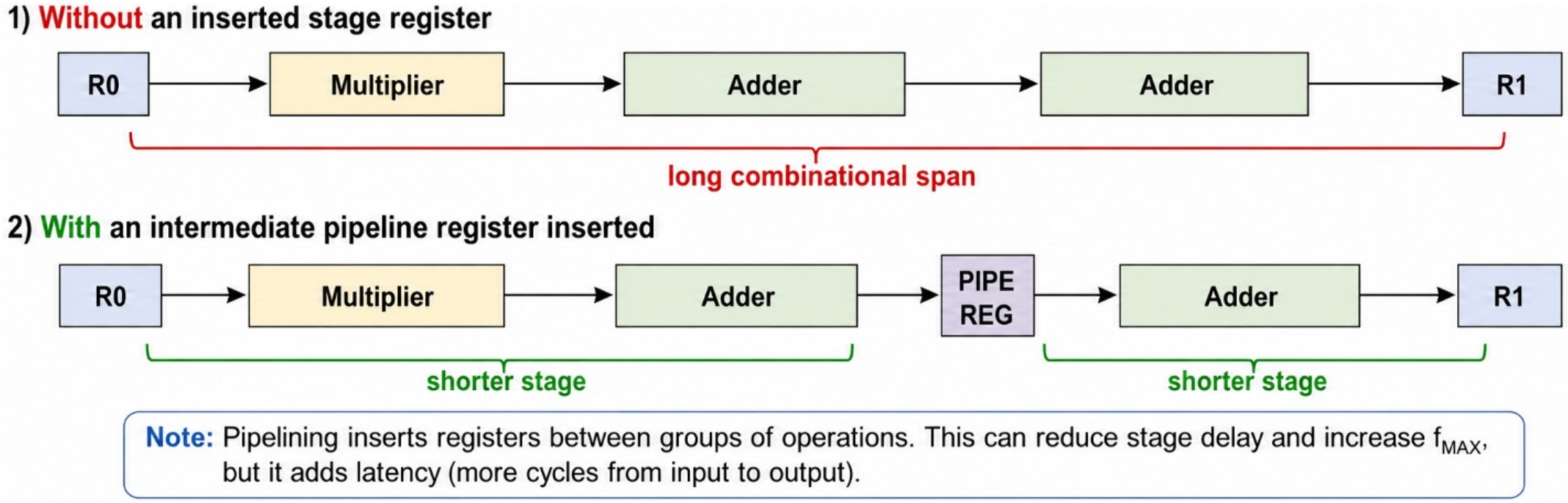
For the register-to-register case shown, the critical setup path is the timing path with the worst setup slack; the red bracket highlights its combinational portion.



*A correct simulation result **does not** prove timing closure.*

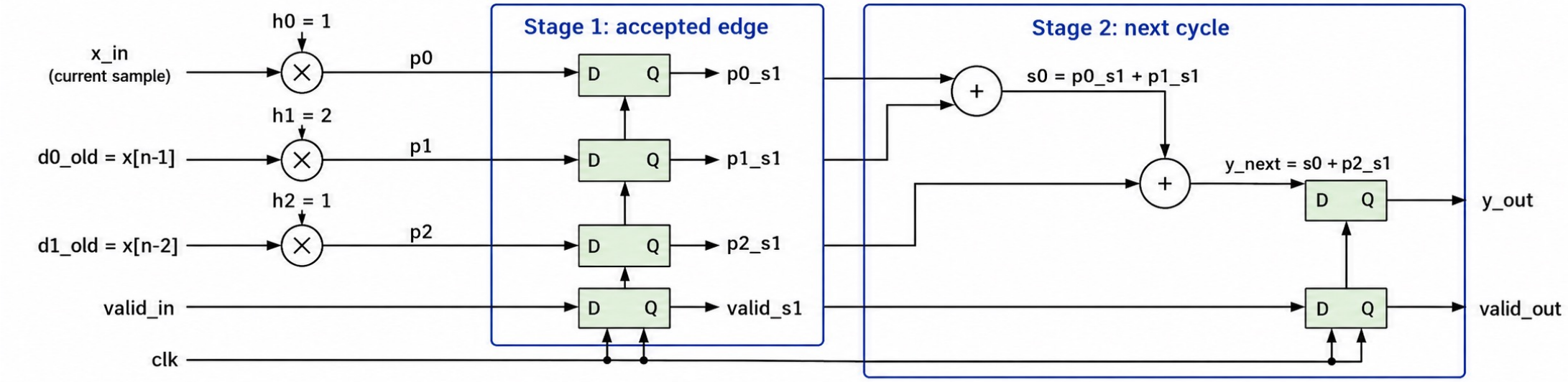
What Pipelining Means

- Pipelining inserts registers between groups of operations
- Each pipeline stage does part of the work
- Well-placed/balanced pipeline stages can reduce the combinational delay per stage
- The output may appear later because of pipeline latency
- Throughput can still be high after the pipeline fills



FIR With Pipeline Stages

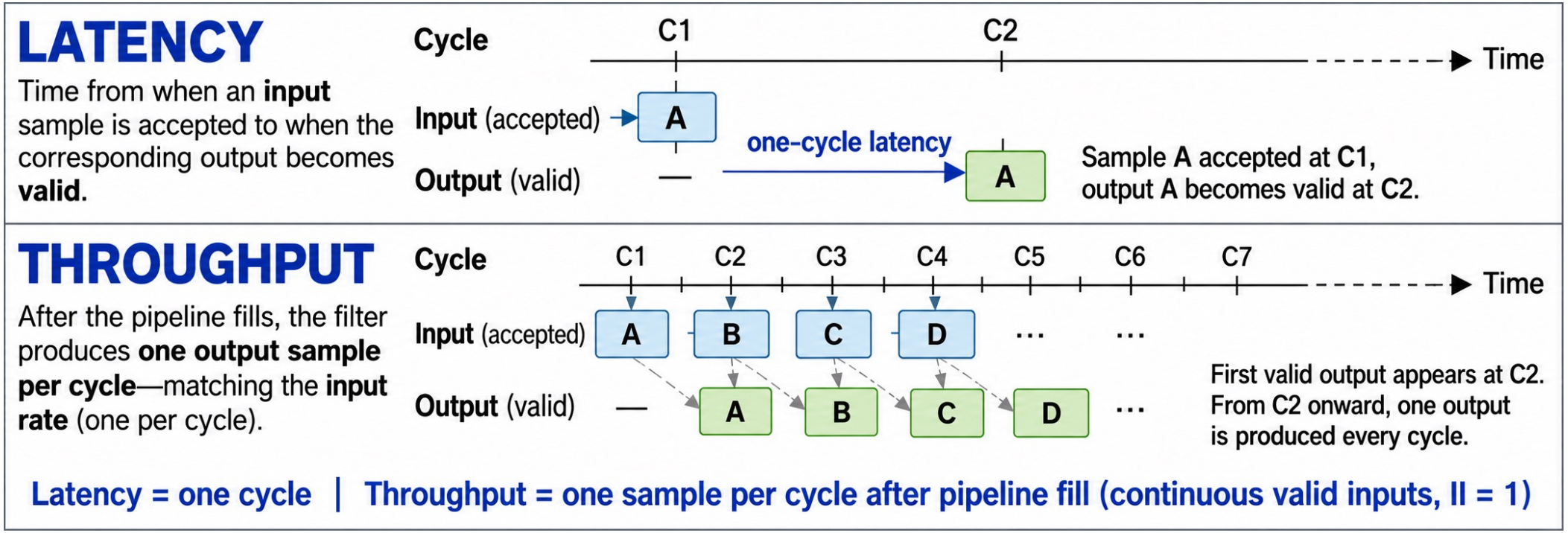
- One stage can compute tap products
- A later stage can add the stored products
- The output corresponds to an earlier input sample
- The testbench must account for this latency
- Waveform reading should connect input sample time to output time



- Accepted edge: x_in, d0_old, d1_old -> p0_s1, p1_s1, p2_s1
- Delay-line update on acceptance: d0_new = x_in, d1_new = d0_old, d2_new = d1_old
- Next cycle: valid_out = 1 and y_out corresponds to the sample accepted one cycle earlier
- One-cycle pipeline latency; after fill, throughput can be one result per accepted sample

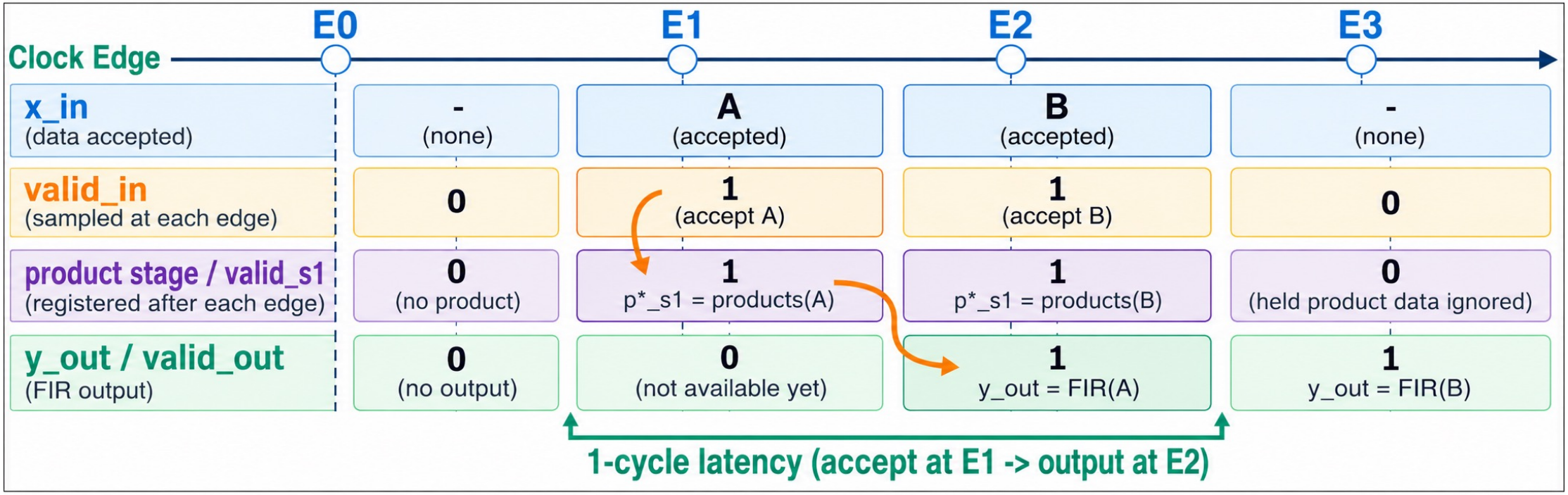
Latency vs Throughput

- Latency is the time from input acceptance to output availability
- Throughput is the rate of completed outputs after steady operation
- A pipelined design often increases latency
- Pipelining can improve throughput in results/second if shorter stages allow a higher clock frequency
- Students must not confuse these two metrics



Valid Signals Across Pipeline

- A valid bit marks when data is meaningful
- Pipeline stages must move valid bits with their data
- Output valid should not assert before the output is correct
- A missing valid delay is a common beginner bug
- The waveform should prove valid/data alignment



Comparing FIR Versions

- Reuse-oriented FIR emphasizes arithmetic hardware reuse across multiple cycles
- Parallel FIR emphasizes one-output-per-sample datapath structure
- Pipelined FIR emphasizes timing improvement through intermediate registers
- All versions should produce the same mathematical FIR outputs
- Their timing, latency, and resource behavior may differ

	Week 4 Sequential Baseline	Week 5 Parallel Registered	Week 5 Pipelined
Arithmetic	Tap products concurrent	Tap products concurrent	Tap products concurrent, then registered
State	Clocked delay-line state	Delay state + registered output	Delay + product + output pipeline registers
Output timing	Combinational after accepted state update	Registered on accepted edge	Corresponding output one cycle later
Initiation rate	Up to 1 sample/cycle	Up to 1 sample/cycle	Up to 1 sample/cycle
Timing expectation	Full arithmetic path	Full arithmetic before output register	Shorter registered stages
Main trade-off	Baseline RTL	Registered output structure	Extra FFs + latency, potentially higher (fMAX)

Time-multiplexed reuse is an additional conceptual alternative with fewer replicated arithmetic units but more cycles per result.

Parallel FIR RTL Skeleton

- Declare delay registers for previous samples
- Compute products from current and delayed samples
- Use a sufficiently wide sum signal
- Register y_out and valid_out on clock edges
- Keep reset behavior explicit and predictable

Plain Verilog — key structure

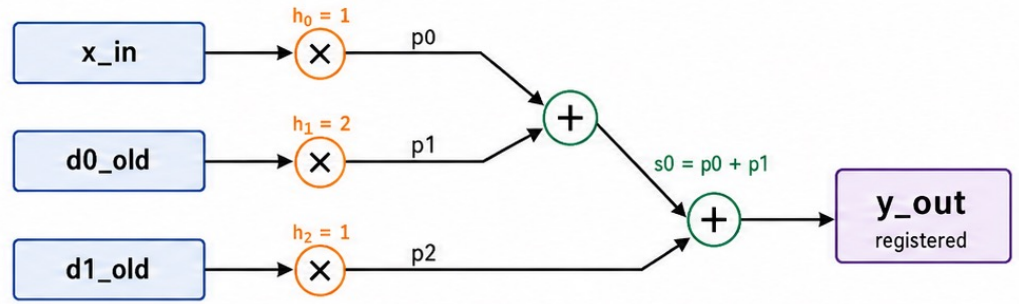
```
reg signed [7:0] d0, d1, d2;
reg signed [9:0] y_out;
reg valid_out;
wire signed [9:0] p0, p1, p2;
wire signed [9:0] s0;
wire signed [9:0] y_next;

assign p0 = {{2{x_in[7]}}, x_in};
assign p1 = {d0[7], d0, 1'b0};
assign p2 = {{2{d1[7]}}, d1};
assign s0 = p0 + p1;
assign y_next = s0 + p2;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        d0 <= 0; d1 <= 0; d2 <= 0;
        y_out <= 0; valid_out <= 1'b0;
    end else if (valid_in) begin
        d2 <= d1; d1 <= d0; d0 <= x_in;
        y_out <= y_next;
        valid_out <= 1'b1;
    end else begin
        valid_out <= 1'b0;
    end
end
```

Same accepted edge

pre-edge values feed arithmetic; registers update together



accepted edge

d0_new = x_in • d1_new = d0_old • d2_new = d1_old

valid_in = 0 → delay line holds; valid_out = 0

Pipelined FIR RTL Skeleton

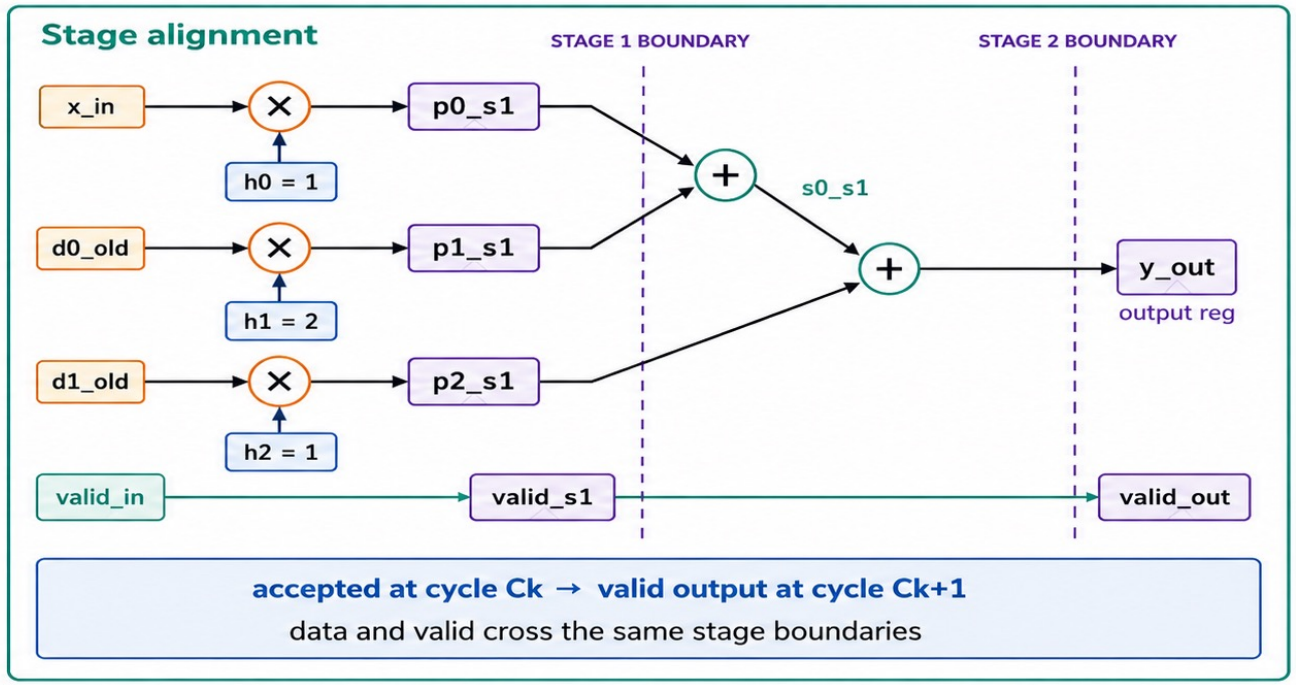
- Stage 1 stores products and a delayed valid bit
- Stage 2 adds stored products and produces output
- Pipeline registers must be updated with nonblocking assignments
- Output valid is delayed relative to input valid
- The testbench checks both values and timing

Plain Verilog — one-cycle pipeline delay

```
reg signed [7:0] d0, d1, d2;
reg signed [9:0] p0_s1, p1_s1, p2_s1;
reg signed [9:0] y_out;
reg valid_s1, valid_out;
wire signed [9:0] s0_s1;
wire signed [9:0] y_s1;

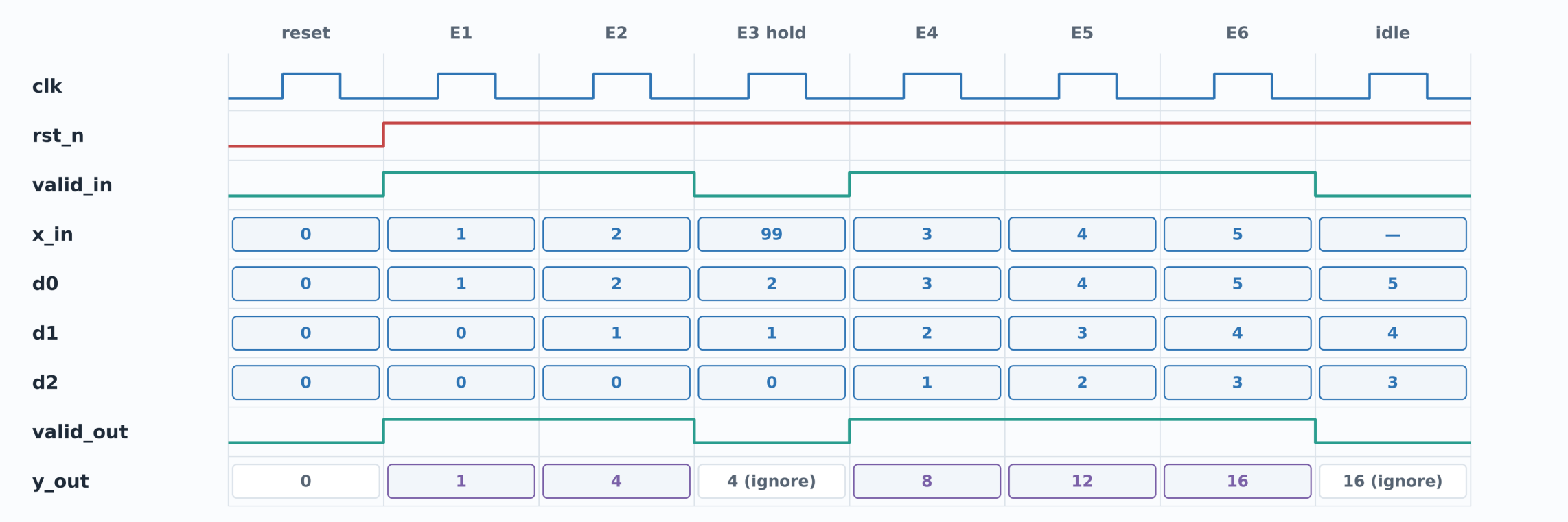
assign s0_s1  = p0_s1 + p1_s1;
assign y_s1   = s0_s1 + p2_s1;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        d0 <= 0; d1 <= 0; d2 <= 0;
        p0_s1 <= 0; p1_s1 <= 0; p2_s1 <= 0;
        y_out <= 0;
        valid_s1 <= 1'b0; valid_out <= 1'b0;
    end else begin
        if (valid_s1) y_out <= y_s1;
        valid_out <= valid_s1;
        valid_s1 <= valid_in;
        if (valid_in) begin
            p0_s1 <= {{2{x_in[7]}}, x_in};
            p1_s1 <= {d0[7], d0, 1'b0};
            p2_s1 <= {{2{d1[7]}}, d1};
    end     d2 <= d1; d1 <= d0; d0 <= x_in;
end
```



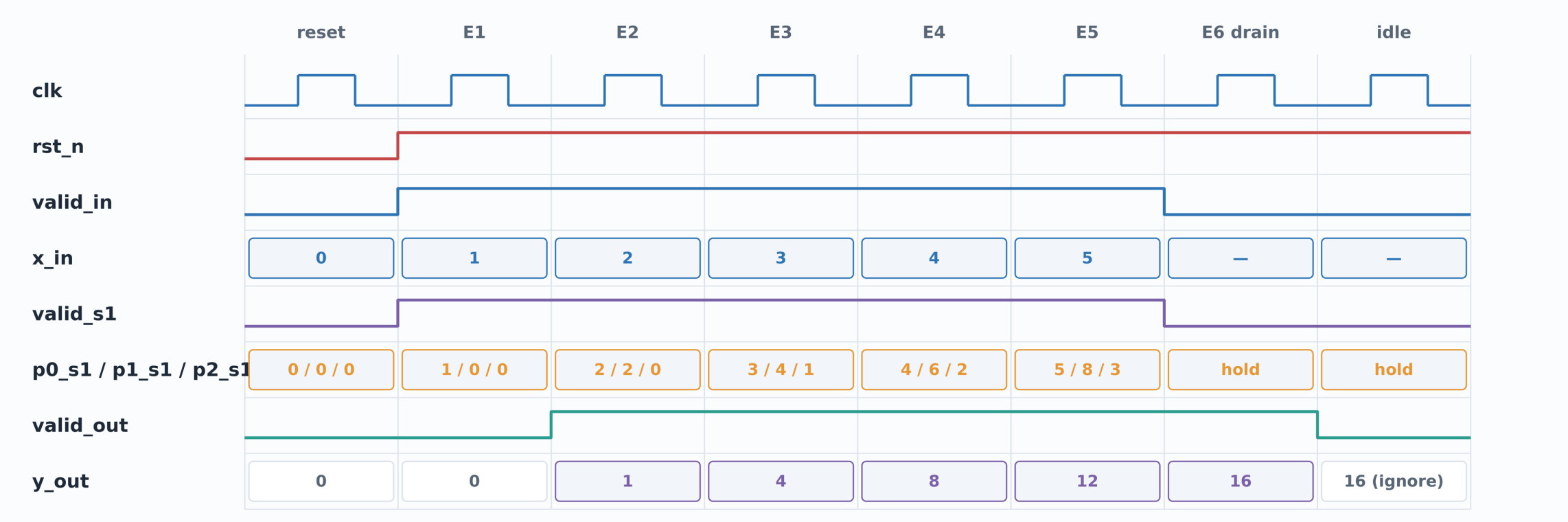
Waveform Evidence: Parallel FIR

- Check clk, rst_n, valid_in, x_in, delay registers, valid_out, and y_out
- Confirm delay registers shift only on rising clock edges with valid_in = 1
- Confirm output appears when valid_out is asserted
- Confirm y_out matches the reference sequence
- Confirm no output is claimed when valid_out is low



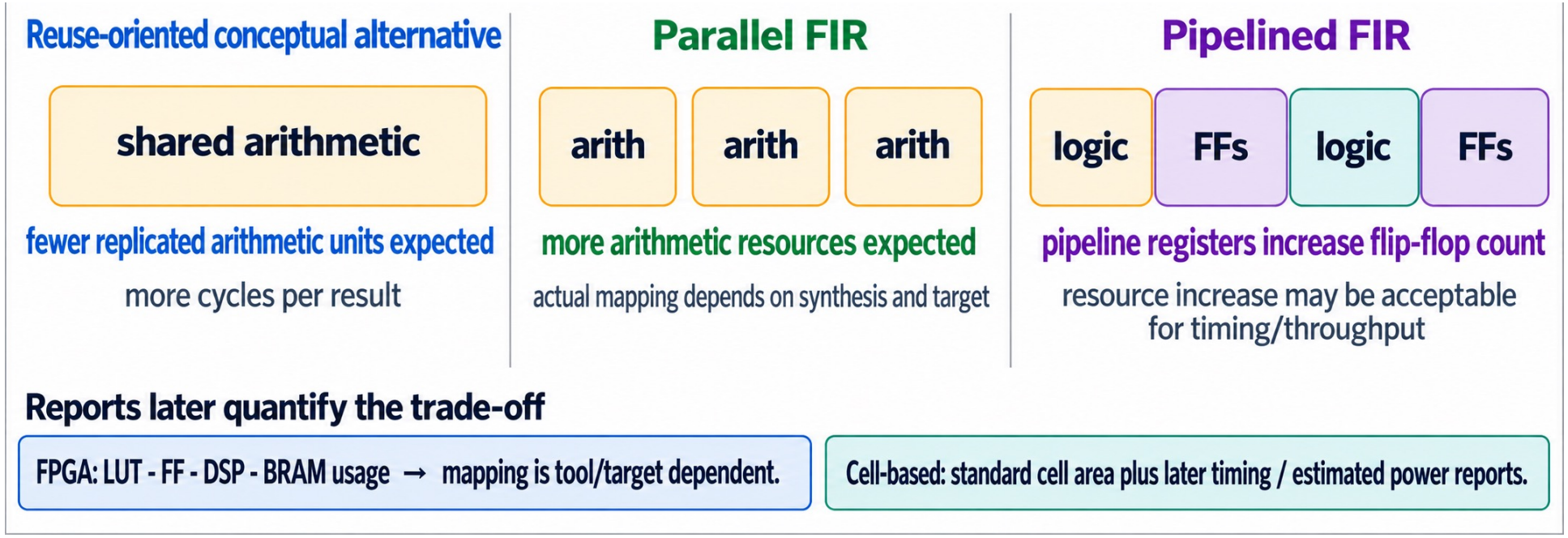
Waveform Evidence: Pipelined FIR

- Input and output occur at different cycles
- The first accepted sample does not produce an immediate valid output
- Pipeline product registers hold intermediate values
- valid_out should align with the delayed sum
- Waveforms should prove latency, not just numerical correctness



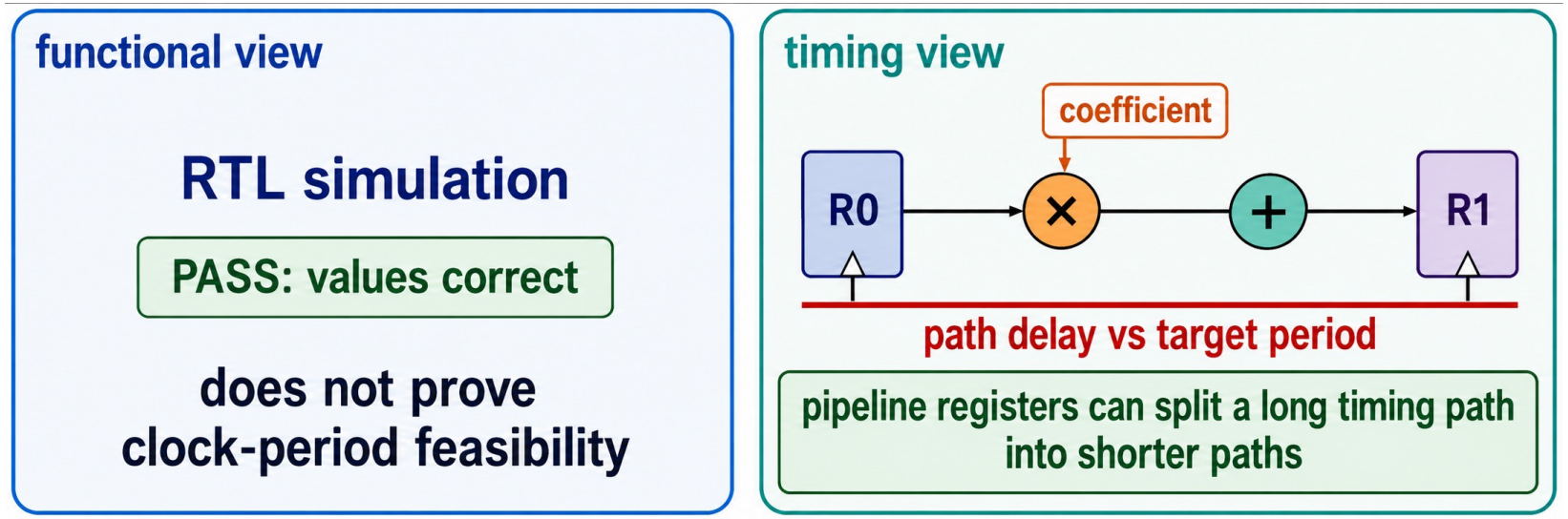
Area and Resource Expectations

- Compared with a time-multiplexed reuse architecture, a fully parallel datapath usually uses more arithmetic resources
- Pipeline registers increase flip-flop count
- Resource increase can be acceptable if timing or throughput improves
- FPGA reports will later show LUT, FF, DSP, and BRAM usage
- Cell-based reports will later show standard-cell area



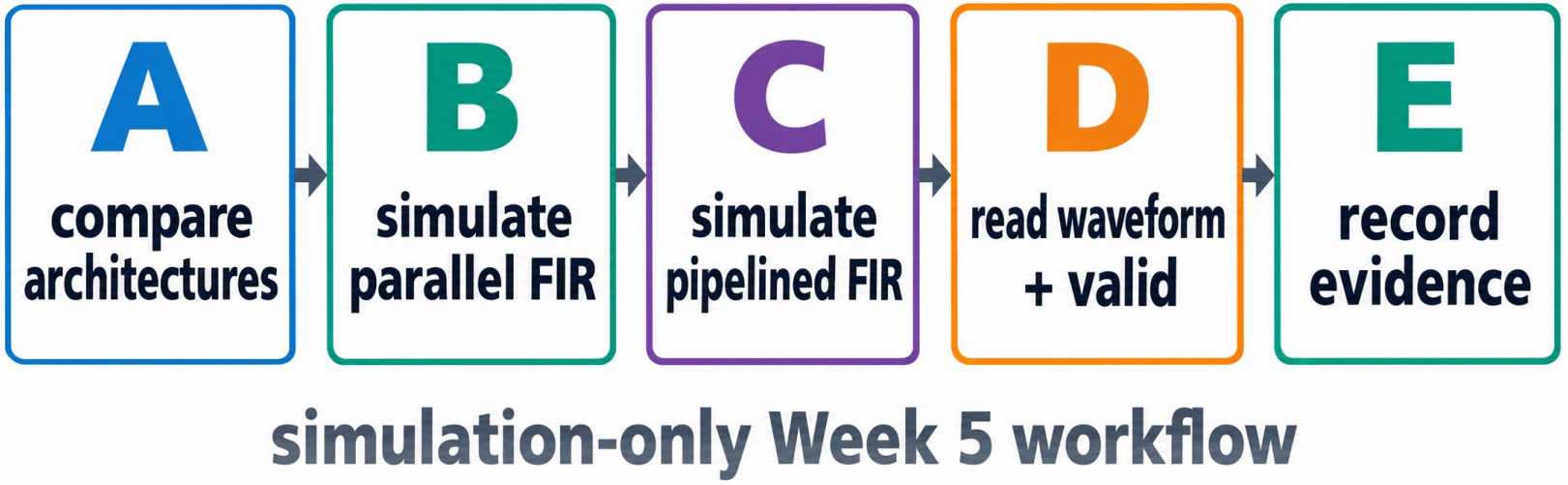
Timing Expectations

- Long combinational paths reduce achievable clock speed
- Well-placed pipelining can reduce per-stage combinational delay
- Timing reports later quantify slack and critical paths
- A design can simulate correctly and still be too slow
- Week 5 prepares students to understand this difference



Lab Activity Overview

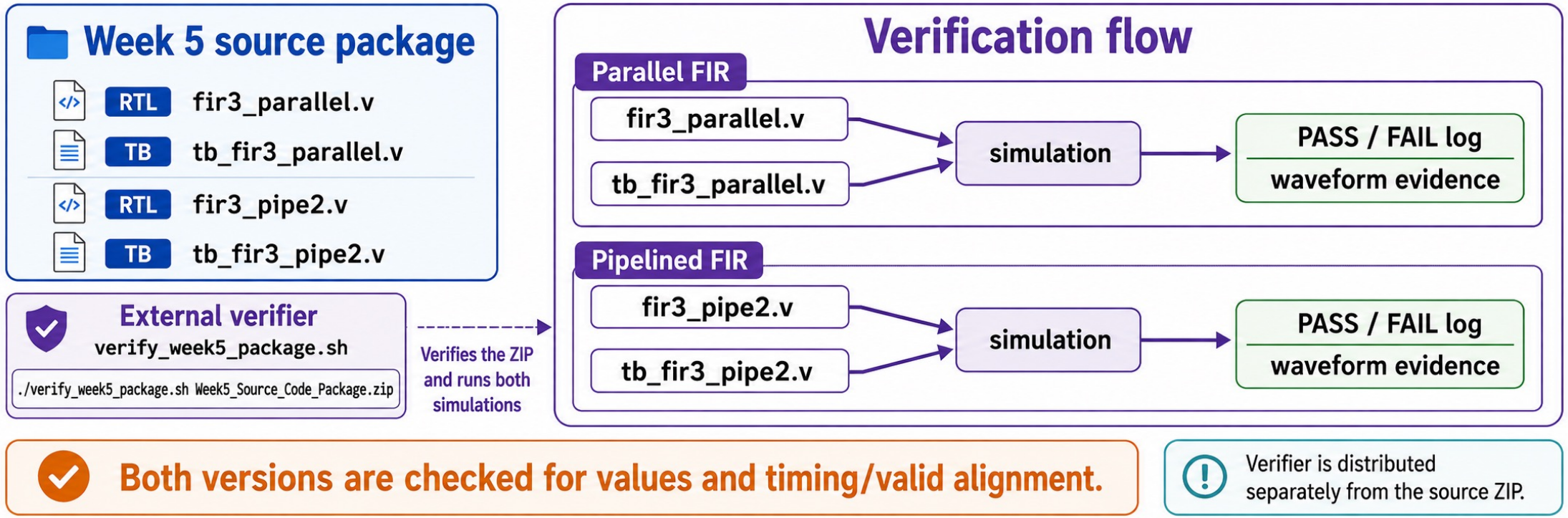
- Part A compares the Week 4 sequential baseline, parallel FIR, and pipelined FIR, with time-multiplexed reuse as an additional architectural trade-off
- Part B simulates the parallel FIR example
- Part C simulates the pipelined FIR example
- Part D reads waveforms and tracks valid signals
- Part E records evidence and explains the design trade-off



Source Package Overview

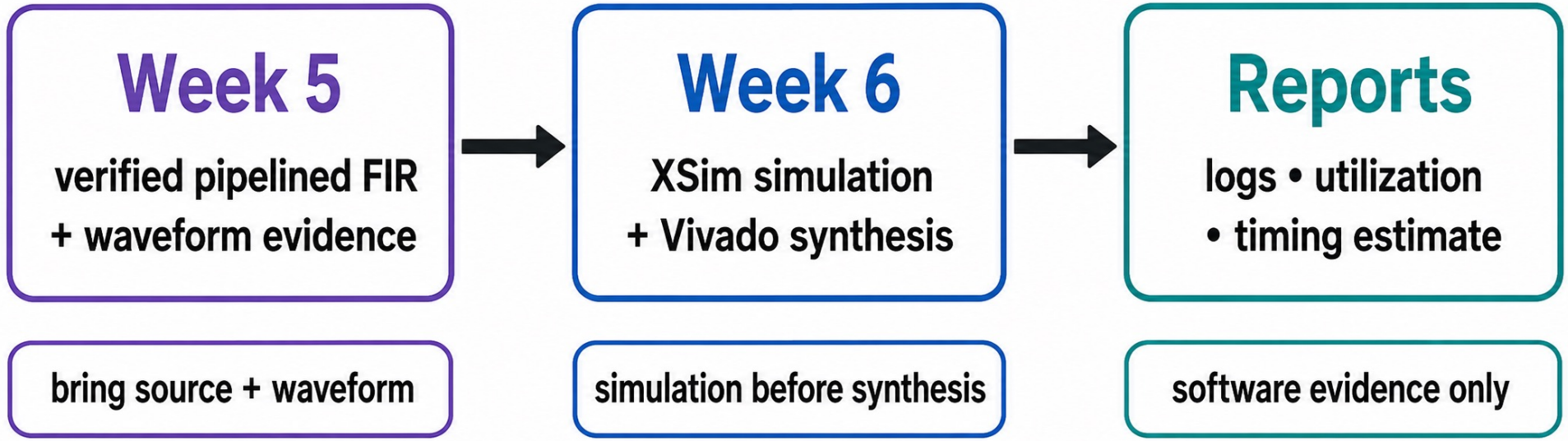
- fir3_parallel.v implements a parallel 3-tap FIR datapath
- tb_fir3_parallel.v checks the parallel FIR behavior
- fir3_pipe2.v inserts a simple pipeline stage
- tb_fir3_pipe2.v checks the delayed valid/output behavior
- The verification script checks PASS/FAIL results automatically

Week 4 baseline: use prior source/waveform as comparison reference; no rerun required.



Prepare for Week 6

- Review the difference between simulation and synthesis
- Review valid signal timing for sequential and pipelined designs
- Bring the Week 5 source package and waveform evidence
- Be ready to run Xilinx FPGA simulation and synthesis steps
- Software-generated evidence remains the focus through Weeks 6–7; Week 8 later adds guided hardware-observation evidence



Week 5 Summary

- Parallelism uses concurrent hardware to do more work at once
- Pipelining inserts registers to split long combinational paths
- Latency and throughput are different performance ideas
- Valid signals must align with data in every design version
- Week 6 begins the Xilinx FPGA flow using these RTL foundations

